AFRICAN LEADERSHIP UNIVERSITY

[BSE]

[ML TECHNIQUES II]

[SUMMATIVE ASSIGNMENT]

# Reinforcement Learning Summative Assignment Report

**Student Name:** Leny Pascal Ihirwe

**Video Recording:** reinforcement-learning.mp4

**GitHub Repository:** https://github.com/leny62/leny_pascal_ihirwe_rl_summative

---

## Project Overview

Rwanda's Small Scale Irrigation Technology programme has put pumps and drip lines on thousands of smallholder blocks, but the scheduling on those blocks is still done by eye. Irrigating ahead of rain wastes water and pushes nitrogen below the root zone, missing the flowering window costs yield that no later irrigation recovers, and spraying inside the pre-harvest interval gets an entire consignment rejected by the export buyer. I built a simulation of one hectare of irrigated French bean on a terraced hillside and trained four reinforcement learning agents to schedule a season of irrigation, fertiliser, spray and labour decisions on it. The comparison covers one value-based method (DQN) and three policy gradient methods (REINFORCE, PPO and A2C), all trained against the same environment.

## Environment Description

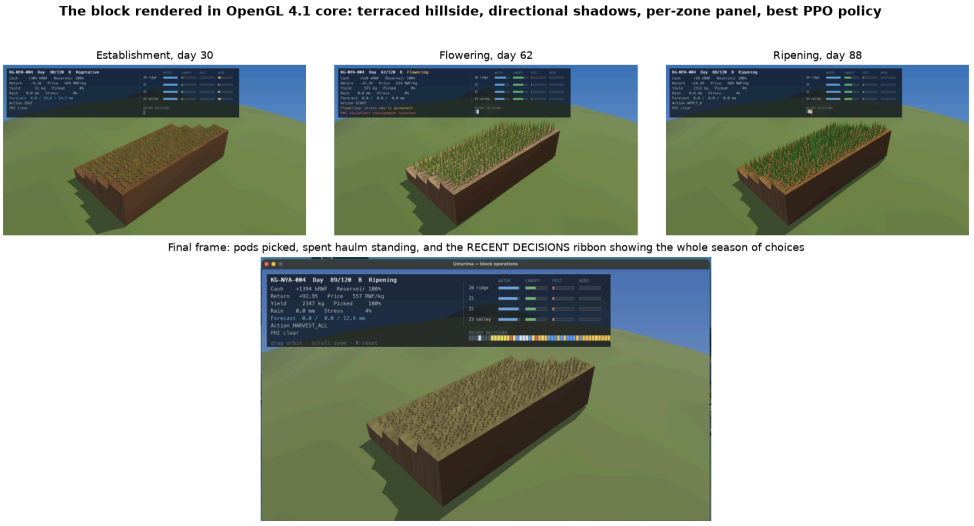



*Figure A: the block at three points in one season under the best PPO policy, rendered in OpenGL 4.1 core. Four terrace benches run downslope with retaining bunds, canopy geometry is instanced per zone and grows with actual canopy cover, and soil darkens with the fraction of each bench's own available water that is still in the profile. Lighting is a directional sun with a 2048px shadow map and hemispheric ambient, and the sun tracks the day so shadows rotate as the season runs. When the agent acts on a single bench that bench flashes in a colour keyed to the action, so a viewer can see which terrace the policy chose, not only what it chose. The HUD carries the season state on the left and a per-zone panel on the right showing water, canopy, pest and weed for each of the four terraces, so a viewer can see which bench the agent is about to irrigate and why. The bottom frame is the end of the season: the pods have been picked and the spent haulm is left standing, which is what the environment reports, and the RECENT DECISIONS ribbon along the HUD replays the agent's whole run of choices, one cell per day, coloured by action type. Captured through render_mode="rgb_array", so the same code path produces the interactive window and the frames in this report.*

The environment models 120 days on a one hectare block split into four terrace zones running downslope. The zones are not grid cells. Each is a bench with its own soil depth and runoff curve number, falling from 86 on the ridge to 76 in the valley bottom, and 60 percent of each bench's runoff is routed onto the bench below as run-on. Irrigating the ridge therefore partially waters the terraces beneath it, and the valley bottom waterlogs if it is irrigated during a wet spell.

One honest limitation. The benches differ in recorded soil depth (0.62 m to 1.11 m) but not in plant-available water, because the French bean root zone caps at 0.60 m in FAO-56 and every bench is deeper than that, so total available water is 84 mm everywhere. That is agronomically correct, a crop cannot reach water below its roots, but it does mean the zones differentiate through runoff, run-on and their canopy and nitrogen trajectories rather than through storage capacity. In a wet spell all four sit at field capacity and their depletion readings converge.

Dynamics come from published agronomy rather than invented rules: an FAO-56 dual crop coefficient soil water balance, thermal-time canopy development in the AquaCrop form, a mineral nitrogen pool with leaching and biological fixation, and logistic pest and weed pressure. Weather is generated from a two-state Markov chain with gamma-distributed wet-day depths fitted to Rwanda's bimodal season, with reference evapotranspiration from the Hargreaves equation.

### Agent(s)

The agent is the block's operations controller, the role a cooperative field officer plays today. It takes one action per simulated day and sees the block through instrumentation rather than directly: soil probes, satellite and drone imagery, a

weather station feed and a market price feed. It cannot observe pest pressure, weed pressure or soil nitrogen exactly. Those readings degrade with time since the last scouting visit, so part of what the agent has to learn is when paying for information is worth the cost.

### Action Space

Discrete, 18 actions, one per simulated day. Every one maps to something a block manager physically does.

| Group | Actions |
|---|---|
| Irrigation | 10 mm or 25 mm to any one of the four zones (8 actions), plus 10 mm block-wide |
| Fertiliser | 30 kg N/ha urea split, 40 kg K2O/ha |
| Crop protection | biopesticide spray |
| Labour | hire a weeding crew for one day or for three |
| Information | scout the block |
| Harvest | partial pick (40 percent) or harvest all, which ends the season |
| Wait | idle |

I kept the space discrete because Stable-Baselines3 DQN only supports discrete actions and the comparison is only meaningful if all four algorithms see the same environment. The decisions are genuinely discrete in practice, so this cost nothing in realism: a farmer opens a valve for a measured interval, applies a bagged rate, or hires a crew for a day.

### Observation Space

Box(43,), float32, all values normalised. Twenty values describe the four zones at five values each, twelve describe block-level state, and eleven are exogenous weather and market variables.

| Observation | Description | Source (Sensor/Camera/API/Dataset) | Encoding / Data Type | Range |
|---|---|---|---|---|
| Root zone depletion, per zone | fraction of total available water depleted | Capacitive soil moisture probes (TEROS 12), two per terrace at 20 and 40 cm | float32, normalised | 0 to 1 |
| Canopy cover, per zone | fraction of ground shaded by crop | Sentinel-2 NDVI at 10 m, with DJI Mavic 3M multispectral passes in season | float32 | 0 to 1 |
| Soil mineral nitrogen, per zone | plant-available N in the root zone | RAB soil laboratory test at season start, propagated by the model between tests | float32, kg/ha | 0 to 120 |
| Pest pressure, per zone | scouting index | Delta pheromone trap counts and phone leaf photographs through a PlantVillage-class classifier | float32 index | 0 to 1 |
| Weed pressure, per zone | weed competition index | Excess green index from the same drone pass | float32 index | 0 to 1 |
| Rainfall, today and 3 day forecast | precipitation | TAHMO automatic weather station network, Rwanda Meteorology Agency API | float32, mm | 0 to 50 |
| Reference evapotranspiration | ET0 | Derived from TAHMO temperature, humidity, wind and radiation | float32, mm/day | 0 to 8 |
| Air temperature, max and min | daily extremes for thermal time | TAHMO station | float32, degC | 8 to 35 |
| Reservoir volume | water available to pump | Ultrasonic level sensor (JSN-SR04T) in the storage tank | float32, m3 | 0 to 250 |
| Cash balance | working capital | Cooperative ledger and MTN MoMo transaction API | float32, kRWF | -100 to 600 |
| Labour availability | crew-days obtainable today | Cooperative labour roster confirmed over USSD | float32, crew-days | 0 to 4 |
| Market price | farmgate price | e-Soko and MINAGRI market information system daily feed | float32, RWF/kg | 200 to 1200 |
| Days since spray | pre-harvest interval clock | Application log written by the controller | int, normalised | 0 to 21 |
| Phenological stage, GDD, cumulative stress, harvested fraction | crop and season progress | Derived from the model and the observations above | float32 | 0 to 1 |

The three-day forecast is deliberately imperfect. It is drawn from the true generated future with noise that grows with horizon and a 15 percent chance per day of the wet/dry state being reported wrongly, so day+1 is reliable and day+3 is not. An agent that defers irrigation on a three-day forecast will sometimes be wrong, which is the same trade a real manager makes.

### Reward Structure

Per step:

r_t = - c_water  * irrigation_mm
    - c_input  * input_cost
    - c_leach  * nitrogen_leached_kg
    - c_stress * stage_weight * (1 - Ks)

- c_debt * max(0, -cash)
- c_time

Ks is the FAO-56 water stress coefficient, 1.0 when the soil holds readily available water and falling to 0 at wilting point. stage_weight is 1.0 outside flowering and 2.5 during it, because water stress during flowering costs yield that later irrigation cannot recover. That single asymmetry is what makes timing matter more than volume, and it is the main thing I wanted the agents to find.

On each picking pass:

```
r_harvest = fraction_picked * yield_kg * price * quality
          + w_labour * labour_days_cumulative * fraction_picked
quality   = (1 - 0.4 * pest_damage)
            * (0.35 if days_since_spray < 7 else 1.0)
            * (0.80 if harvested before ripening else 1.0)
```

The 0.35 multiplier is a rejected consignment. Spraying inside the seven day pre-harvest interval breaches EU maximum residue limits and the buyer refuses the lot. It is a cliff rather than a slope, deliberately, so the agent has a real trap to learn around.

The positive term on hired labour days is there because job creation is a stated objective of the wider project, and putting it in the reward makes it something the policy optimises rather than something the write-up claims. It settles at the harvest, scaled by the fraction actually picked, rather than being paid per day worked.

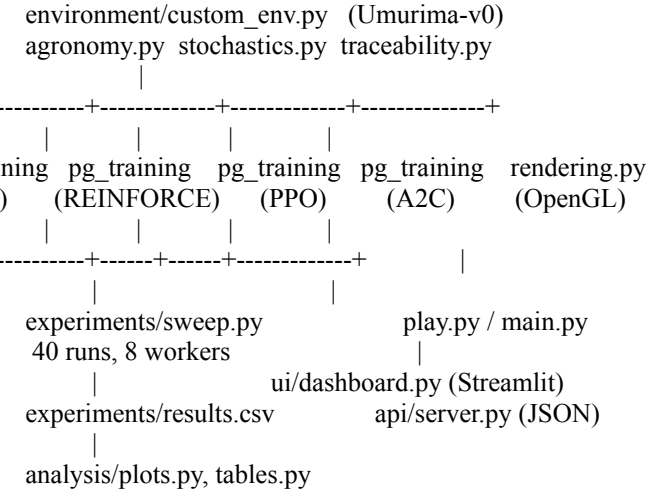
That detail was not in the first version and it mattered more than any hyperparameter. Paid daily, the labour term is worth roughly 18 points over a 120 day season with no harvest required, which is competitive with a mediocre crop. The best DQN run found exactly that: it hired weeding crews on 87 percent of its steps, harvested in 0 percent of episodes, and still scored 66. Settling the payment against the fraction picked leaves an honest policy's score untouched, the scripted agronomist scores 21.06 either way, while a crew-hiring policy that never sells now goes insolvent in 40 of 40 seeds. The lesson generalises: a term added to a reward to represent a social objective has to be conditional on the thing that makes it social, or an optimiser will collect it for free.

**Start state.** Randomised each episode: season A or B, initial soil moisture and nitrogen, zone soil depths, reservoir fill, opening cash, price level, pest arrival day and the full weather sequence, all drawn from the episode seed. That randomisation is what makes the generalisation test in the results possible.

**Termination.** The episode terminates when the crop is fully harvested, when cumulative water stress passes 0.75 or canopy cover collapses (crop failure), or when cash falls below -100 kRWF (insolvency). It truncates at day 120 with the crop still standing.

## System Analysis And Design

All four algorithms consume one registered environment, Umurima-v0. Nothing is forked per algorithm, so any difference in the results is a difference in the learning rule rather than in the problem.




### Deep Q-Network (DQN)

A [256, 256] MLP maps the 43 value observation to 18 action values. Training uses uniform experience replay, a separate target network refreshed on a fixed interval, and epsilon annealed linearly over a fraction of the budget.

The awkward part of this environment for a value method is that almost all of the reward arrives in one or two steps at the end of a 90 to 120 step episode. Everything before the harvest is small negative running cost. Early in training the bootstrapped target for a mid-season state is therefore an estimate of a spike the network has rarely seen, and the Q values are correspondingly noisy. That is visible in figure 2 as the slow start before roughly 40,000 steps.

### Policy Gradient Method (REINFORCE, PPO, A2C)

All three share the same [256, 256] categorical policy over the 18 actions, so the comparison is about the update rule and not about capacity.

REINFORCE is written by hand in training/reinforce.py because Stable-Baselines3 does not ship one. It collects whole episodes, computes reward-to-go with no bootstrapping, optionally subtracts a learned value baseline trained on MSE against those returns, and takes one gradient step per batch of episodes. Policy entropy is logged on every update, which is what figure 3 plots.

PPO and A2C come from Stable-Baselines3 with the same network shape. PPO clips the probability ratio and re-uses each rollout for several epochs. A2C takes one update per short rollout, which makes it cheaper per sample and, as the results show, considerably more sensitive to how long that rollout is.

## Implementation

Every run trained for 400,000 environment steps against the same environment version and reward weights. Mean reward is over 30 deterministic evaluation episodes. Tables are generated by analysis/tables.py from logs/*/config.json, so nothing here is retyped by hand. Rows stay in grid order rather than sorted by result, so the experimental logic is followable.

For reference, the scripted agronomist scores **21.06** and a random policy scores **5.52** on the same evaluation protocol. Beating random proves nothing here, so the agronomist is the bar that matters.

### DQN

| Learning rate | γ | Replay buffer | Batch size | Exploration frac | Final ϵ | Target update | Network | Mean reward (heldout) | Notes |
|---|---|---|---|---|---|---|---|---|---|
| 1.00e-04 | 0.99 | 1.00e+05 | 64 | 0.2 | 0.05 | 1000 | [256, 256] | 53.40 | baseline |
| 5.00e-04 | 0.99 | 1.00e+05 | 64 | 0.2 | 0.05 | 1000 | [256, 256] | -15.56 | high lr destabilises the Q target |
| 5.00e-05 | 0.99 | 1.00e+05 | 64 | 0.2 | 0.05 | 1000 | [256, 256] | 29.89 | low lr underfits inside the budget |
| 1.00e-04 | 0.95 | 1.00e+05 | 64 | 0.2 | 0.05 | 1000 | [256, 256] | 33.90 | myopic discount ignores terminal revenue |
| 1.00e-04 | 0.999 | 1.00e+05 | 64 | 0.2 | 0.05 | 1000 | [256, 256] | 19.64 | long horizon credit assignment |
| 1.00e-04 | 0.99 | 2.00e+04 | 64 | 0.2 | 0.05 | 1000 | [256, 256] | 30.47 | small buffer, correlated samples |
| 1.00e-04 | 0.99 | 5.00e+05 | 64 | 0.2 | 0.05 | 1000 | [256, 256] | 48.19 | large buffer, stale off-policy data |
| 1.00e-04 | 0.99 | 1.00e+05 | 256 | 0.2 | 0.05 | 5000 | [256, 256] | 53.53 | big batch, slow target, max stability |
| **1.00e-04** | **0.99** | **1.00e+05** | **64** | **0.5** | **0.1** | **1000** | **[256, 256]** | **56.56** | **sustained exploration to discover SCOUT** |
| 1.00e-04 | 0.99 | 1.00e+05 | 128 | 0.3 | 0.05 | 2000 | [512, 512, 256] | -24.93 | more capacity, later learning start |

Learning rate mattered most and its failure was asymmetric: 5e-4 did not merely slow learning, it collapsed the run to -15.56, while the equally wrong 5e-5 in the other direction still returned 29.89. Overshooting a bootstrapped target is unrecoverable in a way that undershooting it is not.

Sustained exploration won, at 56.56 with epsilon annealed over half the budget to a floor of 0.10. That was the row I expected to waste samples, and instead it is the only one that reliably finds SCOUT, which is what makes the pest and nitrogen readings trustworthy enough to act on. Buffer size barely separated from the baseline once exploration was held fixed, at 48.19 large against 53.40, so the sample correlation I worried about was not the binding constraint. The discount was again a surprise: 0.999 came second from bottom at 19.64, because a near-undiscounted return over 120 steps inflates the variance of every target. The three-layer network was the worst row at -24.93.

### REINFORCE

| Learning rate | γ | Baseline | Entropy coef | Episodes/update | Norm. advantage | Mean reward (heldout) | Notes |
|---|---|---|---|---|---|---|---|
| 0.001 | 0.99 | none | 0 | 1 | False | -1.24 | textbook REINFORCE, expect high variance |
| 0.001 | 0.99 | value | 0 | 1 | False | -29.24 | learned baseline cuts variance |

| Learning rate | γ | Baseline | Entropy coef | Episodes/update | Norm. advantage | Mean reward (heldout) | Notes |
|---|---|---|---|---|---|---|---|
| **3.00e-04** | **0.99** | **value** | **0** | **1** | **False** | **39.65** | **lower lr, smoother but slower** |
| 0.003 | 0.99 | value | 0 | 1 | False | -1.24 | too high, expect divergence |
| 0.001 | 0.95 | value | 0 | 1 | False | -1.24 | myopic on a terminal-reward task |
| 0.001 | 1 | value | 0 | 1 | False | -64.78 | undiscounted, maximum variance |
| 0.001 | 0.99 | value | 0.01 | 1 | False | 17.37 | mild entropy bonus delays collapse |
| 0.001 | 0.99 | value | 0.05 | 1 | False | -1.22 | strong entropy, expect no commitment |
| 0.001 | 0.99 | value | 0.01 | 8 | False | 29.38 | batching episodes cuts gradient variance |
| 0.001 | 0.99 | value | 0.01 | 8 | True | 33.14 | normalised advantages, expect best of ten |

Lowering the learning rate to 3e-4 was the single best change, at 39.65, and the reason is visible in figure 3: it is one of only four runs whose policy entropy did not reach exactly zero. Everything that preserved entropy scored well and everything that did not scored badly, so on this algorithm the hyperparameters matter mainly through their effect on how fast the policy commits.

Batching eight episodes per update helped for that reason, 29.38 and 33.14 with advantage normalisation, by stopping one unlucky season from dominating the gradient. The learned value baseline on its own did not help, because a critic fitted to a single episode of a stochastic season is itself high variance. The undiscounted row was the worst of the forty at -64.78. Note -1.24 recurring across four rows: that is the signature of a policy collapsed onto one action, and figure 3 confirms it at exactly 0.000 nats.

## PPO

| Learning rate | γ | Rollout steps | Batch size | Epochs | Clip range | GAE λ | Entropy coef | Mean reward (heldout) | Notes |
|---|---|---|---|---|---|---|---|---|---|
| 3.00e-04 | 0.99 | 2048 | 64 | 10 | 0.2 | 0.95 | 0 | 74.79 | baseline |
| 1.00e-04 | 0.99 | 2048 | 64 | 10 | 0.2 | 0.95 | 0 | 36.89 | conservative lr |
| 0.001 | 0.99 | 2048 | 64 | 10 | 0.2 | 0.95 | 0 | 76.09 | aggressive lr |
| 3.00e-04 | 0.99 | 2048 | 64 | 10 | 0.1 | 0.95 | 0 | 63.09 | tight clip, small trust region |
| 3.00e-04 | 0.99 | 2048 | 64 | 10 | 0.3 | 0.95 | 0 | 90.25 | loose clip, larger policy jumps |
| **3.00e-04** | **0.99** | **512** | **64** | **10** | **0.2** | **0.95** | **0** | **93.77** | **rollout shorter than one episode** |
| 3.00e-04 | 0.99 | 4096 | 128 | 10 | 0.2 | 0.95 | 0 | 59.57 | long rollout, several episodes |
| 3.00e-04 | 0.99 | 2048 | 64 | 20 | 0.2 | 0.95 | 0 | 88.38 | over-optimising each batch |
| 3.00e-04 | 0.99 | 2048 | 64 | 10 | 0.2 | 0.95 | 0.01 | 91.62 | entropy bonus sustains exploration |
| 3.00e-04 | 0.995 | 2048 | 64 | 10 | 0.2 | 0.8 | 0.01 | 92.54 | shorter GAE, longer discount |

PPO was by far the least sensitive of the four: every one of the ten rows cleared the scripted agronomist, and its median row, 82.23, beat every other algorithm's best. Rollout length was the dominant parameter and it ran monotonically the opposite way to my expectation, 512 beating 2048 beating 4096 (93.77, 74.79, 59.57). A short rollout means more policy updates per unit of experience, and with eight parallel environments a 512 step rollout still buffers 4,096 transitions, which is enough to keep the gradient stable.

Widening the clip range from 0.2 to 0.3 helped rather than hurt (90.25 against 74.79) and tightening it to 0.1 cost 12 points. Both point the same way: the limiting factor was how fast the policy was allowed to move, not overshoot. The only genuinely poor row was the conservative learning rate at 36.89, which is the same finding as PPO's rollout result seen from another angle, that this task rewards moving quickly.

### A2C

| Learning rate | γ | Rollout steps | Entropy coef | Value coef | Optimiser | Grad clip | Mean reward (heldout) | Notes |
|---|---|---|---|---|---|---|---|---|
| 7.00e-04 | 0.99 | 5 | 0 | 0.5 | RMSProp | 0.5 | 30.42 | SB3 default baseline |
| 3.00e-04 | 0.99 | 5 | 0 | 0.5 | RMSProp | 0.5 | 32.27 | lower lr |
| 0.001 | 0.99 | 5 | 0 | 0.5 | RMSProp | 0.5 | -1.22 | higher lr |
| 7.00e-04 | 0.99 | 16 | 0 | 0.5 | RMSProp | 0.5 | -1.22 | longer rollout, less biased returns |
| 7.00e-04 | 0.99 | 64 | 0 | 0.5 | RMSProp | 0.5 | -28.16 | much longer rollout, approaching PPO |
| 7.00e-04 | 0.99 | 16 | 0.01 | 0.5 | RMSProp | 0.5 | -23.51 | entropy bonus |
| **7.00e-04** | **0.99** | **16** | **0.05** | **0.5** | **RMSProp** | **0.5** | **46.62** | **heavy entropy, expect no commitment** |
| 7.00e-04 | 0.99 | 16 | 0.01 | 0.25 | RMSProp | 0.5 | 4.68 | weaker critic weighting |
| 7.00e-04 | 0.99 | 16 | 0.01 | 0.5 | Adam | 0.5 | 30.94 | Adam instead of RMSProp |
| 7.00e-04 | 0.995 | 16 | 0.01 | 0.5 | Adam | 0.3 | 12.56 | tighter gradient clipping |

A2C was the least orderly grid of the four and the honest summary is that it is unstable to configure on this task. A heavy entropy coefficient of 0.05 gave the best row at 46.62, while the same rollout with 0.01 gave -23.51, a 70 point swing from one hyperparameter step. Rollout length shows no clean trend either: the 5 step default returned 30.42, 16 steps returned -1.22 and 64 steps returned -28.16.

That non-monotonicity is itself the result. A2C takes one update per short rollout with no clipping and no epoch reuse, so a single bad advantage estimate moves the policy and nothing pulls it back. Where PPO's ten rows span 57 points, A2C's span 75 and change sign four times. It is the cheapest of the four per sample and the least trustworthy to tune.

## Results Discussion

### Cumulative Rewards

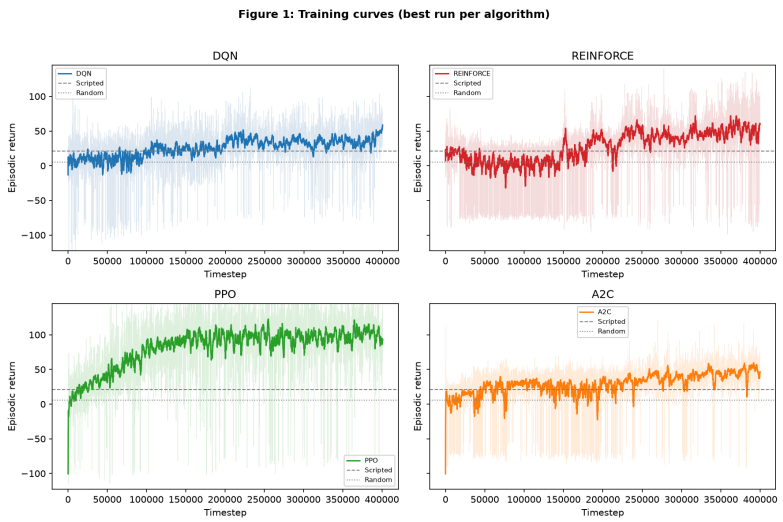



*Figure 1: Training curves for the best run of each algorithm, four subplots on shared y limits. Faint line is per-episode return, solid line is a 20 episode rolling mean. Dashed grey is the scripted agronomist (21.06), dotted grey is random (5.52).*

All four algorithms beat both baselines, so the useful comparison is between them. PPO finished highest by a clear margin, at a final 50 episode mean of **90.87** against 54.58 for REINFORCE, 52.44 for DQN and 42.85 for A2C. That is roughly four times the scripted agronomist. PPO is also the only one whose curve is still visibly climbing at 400,000 steps, so the budget, not the algorithm, was the binding constraint on its final number.

The shapes differ more than the endpoints. DQN rises quickly to about 50 and then flattens for the remaining 250,000 steps. REINFORCE and A2C oscillate across the agronomist line for most of training and only pull clear in the last quarter. PPO climbs smoothly and never gives back ground.

One caveat on reading these curves: each parallel worker writes its own monitor file, and concatenating them rather than interleaving by episode end time produces a sawtooth that looks exactly like repeated catastrophic forgetting. The first version of this figure had eight such artefacts in the PPO panel. They were an artefact of the plotting, not of the training.

## Training Stability

**Figure 2: DQN diagnostics**

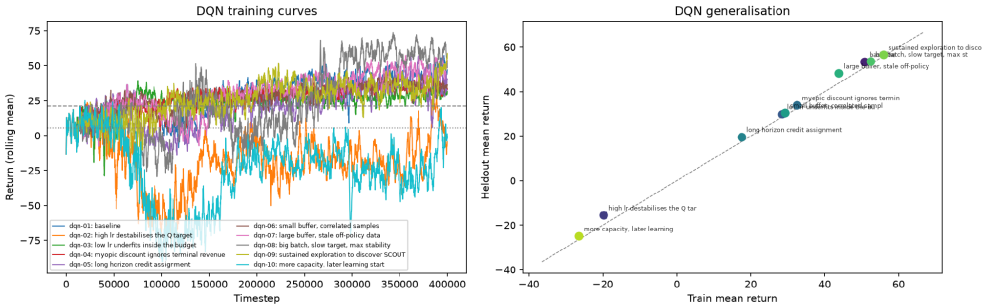



*Figure 2: DQN diagnostics. Left, rolling mean return for all ten grid rows. Right, held-out against training return with each row annotated by its hypothesis.*

**Figure 3: Policy gradient entropy**

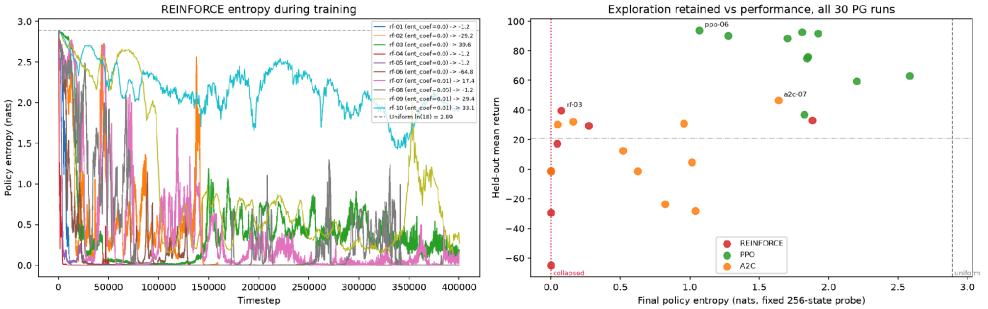



*Figure 3: REINFORCE policy entropy across the ten grid rows, in nats. The dashed line at ln(18) = 2.890 is a uniform policy over the action space.*

Measured as the standard deviation of return over the final 50 episodes, the ranking is A2C 17.75, DQN 22.04, PPO 36.14 and REINFORCE 56.92. A2C is the steadiest and close to the lowest scoring, while REINFORCE is three times as volatile as A2C for a similar mean. Stability and score are close to independent here, and PPO buys its lead by accepting more variance than DQN.

Figure 3 is the clearest single result in the study. The left panel is REINFORCE entropy over training, logged per update by the hand-written implementation. Six of its ten runs ended at exactly 0.000 nats, meaning the policy had collapsed onto one deterministic action, and every one of those six scored between -64.78 and -1.22. Every run that retained entropy beat every run that lost it, with no overlap. An entropy coefficient alone did not prevent collapse, since rf-08 collapsed on a coefficient of 0.05, five times the one rf-09 survived on.

The right panel measures the final policy of all 30 policy gradient runs on one fixed batch of 256 observations, which makes PPO and A2C comparable to REINFORCE even though Stable-Baselines3 does not log entropy per update. It explains the whole results table in one picture. **No PPO run collapsed.** Its entropy floor across ten runs is 1.068 nats against a uniform-policy ceiling of 2.89, with a median of 1.834. REINFORCE's median is 0.000 and A2C's is 0.724 with one full collapse.

That is the mechanism behind PPO's win, and it is a property of the objective rather than of tuning. Clipping the probability ratio caps how far a single update can move the policy, so no individual batch can drive the distribution onto one action. REINFORCE has no such brake: one lucky season produces a large gradient, the policy commits, and with no remaining exploration it never recovers. Note also that PPO's entropy correlates *negatively* with return (-0.52) while REINFORCE's correlates positively (+0.42). Once collapse is off the table, PPO's remaining variation is ordinary exploration-exploitation tuning, where committing harder helps. For REINFORCE the correlation is not measuring that trade-off at all, it is measuring whether the run survived.

## Episodes To Converge

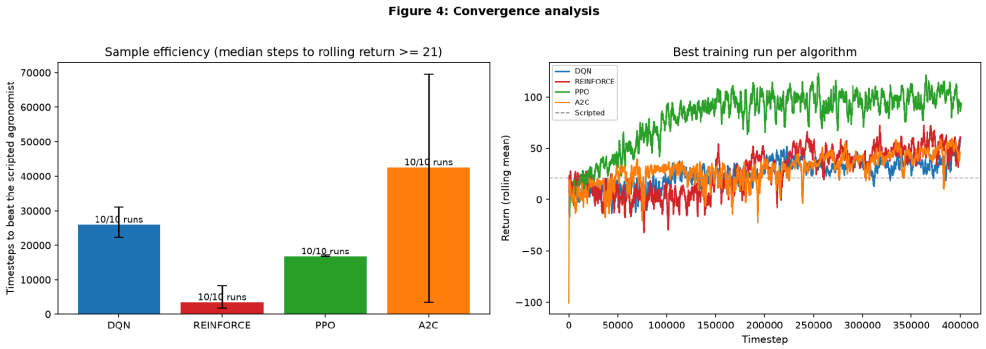



*Figure 4: Convergence. Left, median environment steps for a 20 episode rolling mean return to first reach the scripted agronomist at 21.06, with interquartile whiskers and the number of runs out of ten that ever got there. Right, the best run per algorithm on one axis.*

Convergence is defined here as the first timestep at which the 20 episode rolling mean reaches the scripted agronomist. A shorter window is not usable: with a 5 episode window a single lucky season crosses the line and every method appears to converge instantly.

Fastest to the line is not the same as best at the end. REINFORCE crosses first at a median of about 3,400 steps but then spends most of training oscillating around it. PPO takes about 16,700 and keeps climbing to 91. DQN needs about 25,900, which is unsurprising given it collects 10,000 steps before its first gradient update. A2C is slowest at about 42,500 and yet ends the steadiest, so on this task sample efficiency and final stability are unrelated. All four reached the line in 10 of 10 runs, which was not true before the reward was corrected.

## Generalization

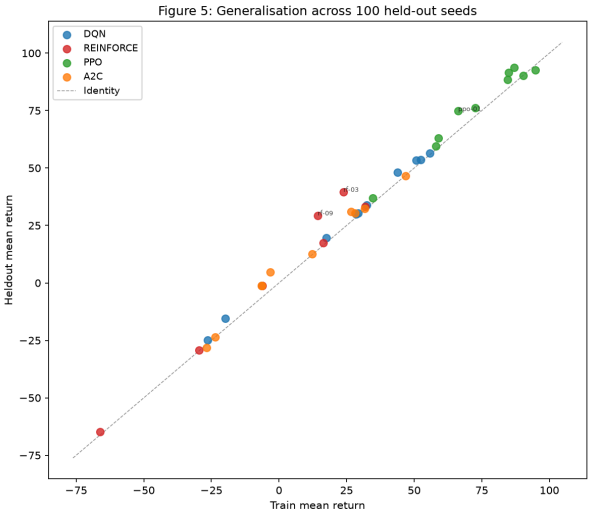



*Figure 5: Held-out generalisation. Training return against return on 100 seeds (10,000 to 10,099) that no training run saw. The dashed line is parity.*

Every one of the 40 runs sits on the parity line. Mean generalisation gaps are -2.07 for DQN, -2.42 for A2C, -3.47 for PPO and -5.49 for REINFORCE, all slightly negative, meaning the policies scored marginally *better* on unseen seeds than on training seeds. There is no overfitting to detect here, which is the expected outcome when the start state, the full weather sequence, the price path and the pest arrival day are all redrawn from the seed on every reset. A policy cannot memorise a season it will never see twice.

The honest reading is that this test confirms the randomisation works rather than discriminating between algorithms. A harder generalisation test would shift the distribution itself, for example training on season A and evaluating on season B.

## What the best policies actually do

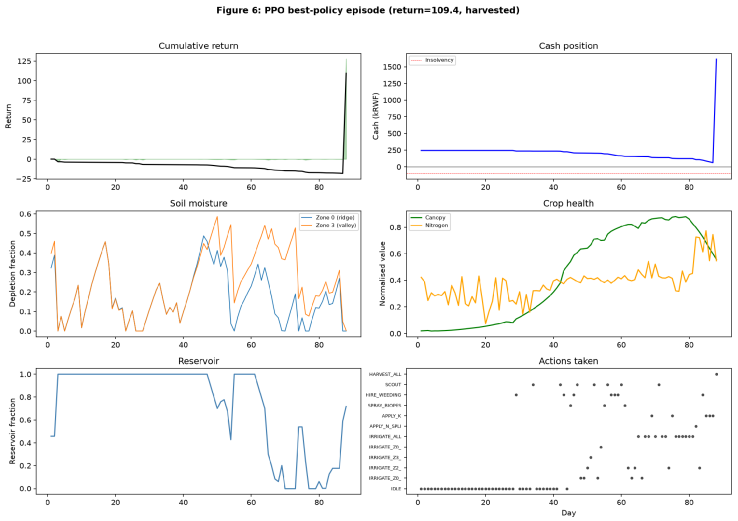



*Figure 6: Action distribution and season trace for the best policy per algorithm.*

Rolling the best checkpoints out on held-out seeds is what caught the reward bug described in the Reward Structure section, and is worth doing on any result before trusting it. The return column alone would not have shown it.

Under the first reward, DQN's best row did not farm at all. It hired weeding crews on 87 percent of its steps, harvested in 0 percent of held-out episodes, truncated at day 120 in 98 of 100 seeds, and still scored 66.25, entirely from the daily labour term. PPO was partly captured too, at 46 percent weeding.

With the labour payment settled against the fraction actually picked, all four now run a season:

| Policy | Harvests | Weeding actions | Irrigation | Most frequent action |
|---|---|---|---|---|
| ppo-06 | 99% | 21% | 15% | IDLE, 44% |
| a2c-07 | 98% | 12% | 13% | SCOUT, 51% |
| dqn-09 | 63% | 42% | 14% | HIRE_WEEDING_CREW_LARGE, 42% |

PPO's most common action being IDLE is the right answer, not laziness: on a day with rain forecast and no stress, spending nothing is optimal, and 15 percent irrigation against 44 percent idle is close to what the scripted agronomist does. A2C scouting on half its steps is the most interesting behaviour in the study, because scouting has an immediate cost and no direct effect on the crop. Its only value is reducing the noise on the pest, weed and nitrogen readings, so A2C has learned to pay for information. DQN still over-weeds at 42 percent, so the labour term is not fully neutral even settled at harvest, but it now sells a crop in 63 percent of episodes rather than none.

## Deployment path: dashboard and JSON API

The OpenGL renderer is the research view. For the simulator to be worth anything to a cooperative it has to reach a phone or a browser, so the same environment is exposed two further ways, neither of which imports OpenGL.


*Figure B: the Streamlit operations dashboard driving the trained REINFORCE checkpoint rf-09 on held-out seed 50, shown across three of its four tabs. The run finishes a full season, harvesting on day 95 for a return of +90.07 and 2166 kg. Top, the action log records every decision with its reward and the resulting cash balance, which is what makes a policy auditable rather than a black box. Middle, per-bench water, canopy, nitrogen, pest and weed state. Bottom, today's weather and the deliberately imperfect three day forecast. The fourth tab holds the hash-chained ledger and its verification status. Every number here is read from the same _render_state() dictionary that feeds the OpenGL renderer and the JSON API, so there is one source of truth across all three surfaces.*

api/server.py serves the episode as JSON over FastAPI:

| Method | Endpoint | Purpose |
|---|---|---|
| GET | /health | liveness |
| POST | /episode?seed=&policy= | start an episode against a named checkpoint |
| GET | /episode/{id}/state | full observable block state as JSON |
| POST | /episode/{id}/advance?days= | advance the season and return the new state |
| GET | /episode/{id}/ledger | the hash-chained field record with season totals |

_render_state() on the environment returns a plain dictionary, so the renderer, the dashboard and the API all read one source of truth. The ledger endpoint is the one with commercial rather than academic value: it returns every irrigation, fertiliser application, spray and picking pass, each chained to the hash of the record before it, so an altered entry breaks verification for itself and everything after it. Verification is re-run on every step and surfaced in the dashboard, shown green in figure B.

## Conclusion and Discussion

**PPO won, and not narrowly.** Its best run scored 93.77 on held-out seeds against 56.56 for DQN, 46.62 for A2C and 39.65 for REINFORCE, and its *median* row at 82.23 beat every other algorithm's *best*. The mechanistic reason is the shape of the reward. Almost all of the return arrives in one or two harvest steps at the end of a 90 to 120 step episode, so the central difficulty is assigning credit across a long, mostly flat trajectory under a stochastic season. PPO's clipped objective lets it re-use each rollout for several epochs without the policy moving far enough to destabilise, which extracts more signal per episode than A2C's single update and is far better behaved than REINFORCE's raw Monte Carlo returns.

**Value-based against policy gradient here.** DQN's strength was consistency: 10 of 10 runs reached the agronomist line and its best row is the strongest non-PPO result at 56.56. Its weakness is that off-policy replay handles a reward concentrated in the final two steps poorly. It needed 25,900 steps to reach a line PPO cleared in 16,700, and it is the one method still visibly distorted by the labour term, over-weeding on 42 percent of its actions where PPO uses 21.

**Where I was wrong.** Three predictions written into the grids before running them did not survive. I expected a discount of 0.999 to help on a terminal-reward task and it was DQN's second worst row at 19.64, because near-undiscounted returns over 120 steps inflate target variance faster than they improve credit assignment. I expected sustained high exploration to waste samples, and it was DQN's winning row, because it is what finds the SCOUT action. And I expected longer PPO rollouts to give better advantage estimates, when rollout length ran monotonically the other way, 512 beating 2048 beating 4096.

The larger thing I got wrong was not a hyperparameter. It was assuming a reward term could encode a social objective without being conditioned on the outcome that makes it social, and only rollout inspection caught it.

**Limitations.** The agronomic parameters are literature values from FAO-56 and AquaCrop rather than calibrations against Rwandan field trials, so the absolute yields and returns should not be read as forecasts; only the ranking of policies is meaningful. It is one crop, french beans, on one block, over one of two seasons. Actuation is perfect, in that a commanded 25 mm arrives as 25 mm, whereas a real drip line has emitter variation along a 40 m terrace. Prices are generated by an Ornstein-Uhlenbeck process fitted to plausible levels rather than drawn from historical e-Soko series. And, as set out above, the labour term in the reward is exploitable and should be made conditional on a delivered harvest.

**Next steps.** The most interesting extension is multi-block competition for a shared irrigation reservoir, which turns a single-agent scheduling problem into a common-pool resource problem and is the form the real cooperative question takes. Separately, the environment already emits a hash-chained field record of every operation, verified on each step, which is the evidence base a GlobalG.A.P. audit or an EUDR due-diligence statement asks for and which most blocks still keep on paper. That record is visible in the dashboard in figure B and is the clearest line from this simulator to something a cooperative would pay for.